

BINARY SEARCH - PAGE 1

Date : 10 / 07 / 2025

Topic : Binary Search

1. INTRODUCTION

Binary Search is an efficient searching algorithm that works on a sorted array by repeatedly dividing the search interval in half.

It compares the middle element with the target value.

If they match, the element is found.

If not, it eliminates the half in which the target cannot exist and repeats the process on the remaining half.

KEY CHARACTERISTICS

- Works only on sorted arrays.
- Much faster than Linear Search.
- Time Complexity : $O(\log n)$
- Reduces search space by half in each step.
- Efficient for large datasets.

Key Idea :

Instead of checking every element one by one like Linear Search ($O(n)$), Binary Search eliminates half of the remaining elements in every step.



2. IDEA / INTUITION

- Since the array is sorted, we can use the middle element to decide where the target can exist.
- If $\text{target} == \text{middle element} \rightarrow \text{Found!}$
- If $\text{target} < \text{middle element} \rightarrow \text{Target must be in the left half.}$
- If $\text{target} > \text{middle element} \rightarrow \text{Target must be in the right half.}$
- Repeat this process on the appropriate half until the element is found or the interval becomes empty.

3. EXAMPLE TO UNDERSTAND

Let the sorted array be :

2	5	8	12	16	23	38	56	72
0	1	2	3	4	5	6	7	8

Let target = 23

Step 1:

2	5	8	12	16	23	38	56	72
low = 0				mid = 4				high = 8

$\text{arr}[\text{mid}] = 16$

$23 > 16 \rightarrow \text{Search in right half}$

New range : index 5 to 8

Step 2:

2	5	8	12	16	23	38	56	72
				low = 5	mid = 6	high = 8		

$\text{arr}[\text{mid}] = 38$

$23 < 38 \rightarrow \text{Search in left half}$

New range : index 5 to 5

Step 3:

2	5	8	12	16	23	38	56	72
				low = 5	mid = 5	high = 5		

$\text{arr}[\text{mid}] = 23$

$23 == 23 \rightarrow \text{Found!}$

Therefore, 23 is found at index 5.

Important Note

Binary Search always reduces the search space by half.

4. ALGORITHM (STEP-BY-STEP)

1. Start with $\text{low} = 0$ and $\text{high} = n - 1$.
2. While $\text{low} \leq \text{high}$, do the following:
 - Calculate $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$
 - If $\text{arr}[\text{mid}] == \text{target}$, return mid .
 - If $\text{target} < \text{arr}[\text{mid}]$, set $\text{high} = \text{mid} - 1$.
 - Else, set $\text{low} = \text{mid} + 1$.
3. If the loop ends, the element is not present. Return -1.

Why $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$?

To avoid integer overflow that can occur with $(\text{low} + \text{high}) / 2$ in some programming languages when low and high are very large.



5. PSEUDOCODE

BinarySearch(arr, target)

low $\leftarrow$ 0

high $\leftarrow$ length(arr) - 1

while low $\leq$ high

mid $\leftarrow$ low + (high - low) // 2

if arr[mid] == target

return mid // Element found

else if target < arr[mid]

high $\leftarrow$ mid - 1 // Search left half

else

low $\leftarrow$ mid + 1 // Search right half

return -1 // Element not found



"Binary Search is all about smarter division, not harder searching." 😊



BINARY SEARCH - PAGE 2

Date : 10/07/2025

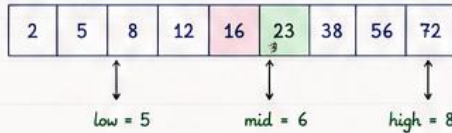
Topic : Binary Search

1. EXAMPLE DRY RUN (CONTINUED)

Array : [2, 5, 8, 12, 16, 23, 38, 56, 72] Target = 23

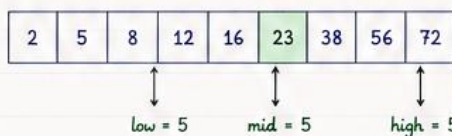
(Index : 0 1 2 3 4 5 6 7 8)

Step 4 :



arr[mid] = 38
23 < 38 → Search
in left half
New range :
index 5 to 5

Step 5 :



arr[mid] = 23
23 == 23 → Found!

Result : Key 23 is found at index 5.



3. ACTUAL CODE (C)

```
#include <stdio.h>
int binarySearch(int arr[], int n, int target) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = low + (high - low) / 2;
        if (arr[mid] == target)
            return mid; // Element found
        else if (arr[mid] > target)
            high = mid - 1; // Search left half
        else
            low = mid + 1; // Search right half
    }
    return -1; // Element not found
}

int main() {
    int arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 23;
    int result = binarySearch(arr, n, target);
    if (result != -1)
        printf("Key %d found at index %d\n", target, result);
    else
        printf("Key %d not found in the array.\n", target);
    return 0;
}
```

5. ADVANTAGES

- ✓ Very fast ($O(\log n)$).
- ✓ Efficient for large sorted datasets.
- ✓ Requires less time and comparisons.
- ✓ Uses less memory in iterative approach.

6. DISADVANTAGES

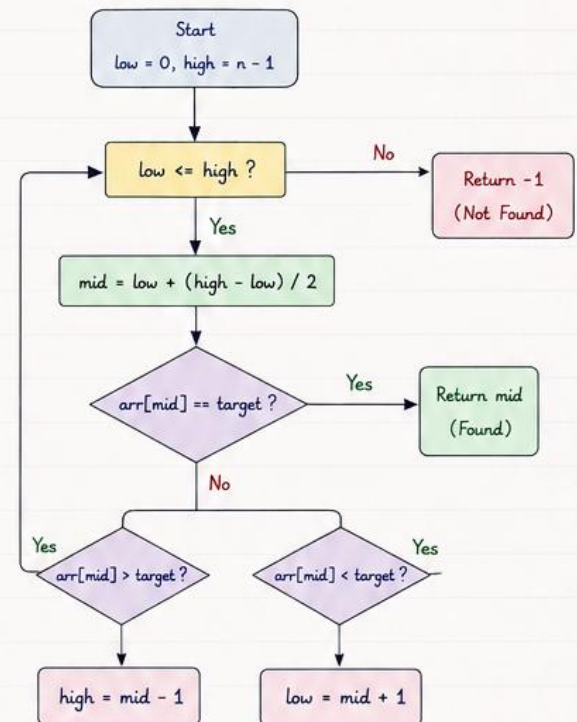
- ✗ Works only on sorted arrays.
- ✗ Not suitable for unsorted data.
- ✗ Insertion/Deletion in arrays can be expensive.

7. APPLICATIONS

- * Searching in large datasets.
- * Used in databases.
- * Dictionary lookups.
- * Finding an element in sorted lists.

2. VISUAL SUMMARY

Step	low	high	mid	arr[mid]	Comparison	Action
1	0	8	4	16	23 > 16	Search Right
2	5	8	6	38	23 < 38	Search Left
3	5	5	5	23	23 == 23	Found!



4. COMPLEXITY ANALYSIS

Case	Time Complexity
Best Case (Target is middle element)	$O(1)$
Average Case	$O(\log n)$
Worst Case (Target not present)	$O(\log n)$
Space Complexity (Iterative)	$O(1)$
Space Complexity (Recursive)	$O(\log n)$

Why $O(\log n)$?

At every step, Binary Search eliminates half of the remaining elements. So the number of steps required is proportional to $\log_2 n$.



"The beauty of Binary Search lies in its simplicity and power." 😊

